

深圳大学实验报告

课程名称: 数值计算方法

实验项目名称: 机械求积、Newton-Cotes 求积与复合求积

学院: 计算机与软件学院

专业: 人工智能卓越班

指导教师: 吴晓群、卯丙

报告人: 邓瑞霖 学号: 2024150040 班级: 01

实验时间: 2026 年 6 月 10 日 -- 2026 年 7 月 1 日

实验报告提交时间: 2026 年 6 月 15 日星期一

教务部制

实验目的与要求:

1. 了解数值积分基本概念、基本思想、代数精度与求积公式稳定性
2. 掌握矩形积分、梯形积分、Simpson 积分等机械求积公式以及插值型求积公式
3. 掌握 Newton-Cotes 求积公式与复合求积公式
4. 上机实现机械求积、Newton-Cotes 求积与复合求积, 并对比分析不同求积算法的精度和效率

基本原理与算法 (详细阐述实验涉及数值计算方法的数学原理及其对应的算法步骤)

1. 机械求积公式

1.1 数值积分的基本思想

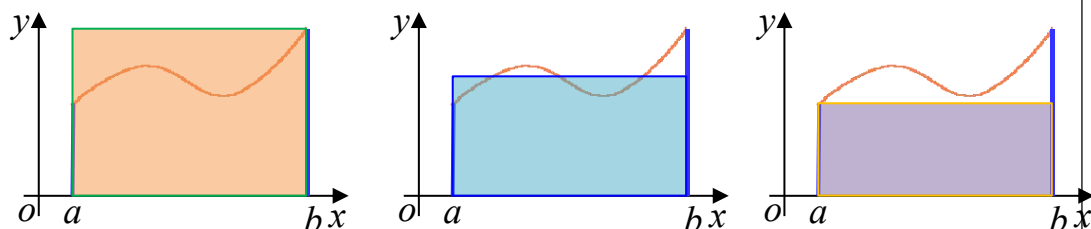
若用直线段

$$y = f(\theta a + (1 - \theta)b), \theta \in [0, 1]$$

近似代替曲线段 $y = f(x) (x \in [a, b])$, 则可得矩形积分公式

$$\int_a^b f(x) dx \approx (b - a) f(\theta a + (1 - \theta)b), \theta \in [0, 1]$$

当 $\theta = 0, \frac{1}{2}, 1$ 时, 分别称为左矩形公式, 中矩形公式, 右矩形公式



用过点 $A(a, f(a)), B(b, f(b))$ 的直线段

$$y = f(a) + \frac{f(b) - f(a)}{b - a}(x - a), x \in [a, b]$$

近似代替曲线段 $y = f(x) (x \in [a, b])$, 则可得梯形积分公式

$$\int_a^b f(x) dx \approx \frac{b - a}{2} (f(a) + f(b))$$

考虑过三点 $A(a, f(a)), C((a + b)/2, f((a + b)/2)), B(b, f(b))$ 的抛物线段 $y = px^2 + qx + r, (x \in [a, b])$

其中 p, q, r 由如下方程组确定:

$$\begin{cases} pa^2 + qa + r = f(a), \\ p\left(\frac{a+b}{2}\right)^2 + q\frac{a+b}{2} + r = f\left(\frac{a+b}{2}\right) \\ pb^2 + qb + r = f(b) \end{cases}$$

若用此抛物线段近似代替曲线段 $y = f(x), (x \in [a, b])$, 则得 Simpson 积分公式:

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right)$$

算法步骤:

- 1) 分割: $[a, b] \ a = x_0 < x_1 < \dots < x_{n-1} < x_n = b$
- 2) 近似代替: 任取 $\xi_i \in [x_{i-1}, x_i]$, 作 $\Delta A_i \approx f(\xi_i) \Delta x_i (\Delta x_i = x_i - x_{i-1}), i = 1, 2, \dots, n$
- 3) 求和: $A = \sum_{i=1}^n \Delta A_i \approx \sum_{i=1}^n f(\xi_i) \Delta x_i$
- 4) 取极限: 令 $\lambda = \max_{1 \leq i \leq n} \{\Delta x_i\}$, 则曲边梯形面积

$$A = \lim_{\lambda \rightarrow 0} \sum_{i=1}^n \Delta A_i = \lim_{\lambda \rightarrow 0} \sum_{i=1}^n f(\xi_i) \Delta x_i$$

矩形法:

$$\int_a^b f(x) dx = \sum_{k=1}^n \int_{x_{k-1}}^{x_k} f(x) dx \approx \sum_{k=1}^n (x_k - x_{k-1}) f(x_k)$$

梯形法:

$$\int_a^b f(x) dx \approx \sum_{k=1}^n \frac{1}{2} (x_k - x_{k-1}) [f(x_{k-1}) + f(x_k)]$$

数值求积公式的一般形式:

$$\int_a^b f(x) dx \approx \sum_{k=0}^n A_k f(x_k)$$

其中, 式中 x_k 称为求积节点; A_k 称为求积系数

1.2 待定系数法

定义 1: m 次代数精度

如果某求积公式对于次数不超过 m 的多项式, 均能精确成立, 但对于 m+1 次多项式不能精确成立, 则称该求积公式具有 m 次代数精度

以代数精度为标准构造求积公式的方法称为待定系数法

一般的, 要使求积公式 $\int_a^b f(x) dx \approx \sum_{k=0}^n A_k f(x_k)$ 具有 m 次代数精度, 只要令它对

于 $f(x) = x^i (i = 0, 1, \dots, m)$ 都能精确成立, 即要求

$$\sum_{k=0}^n A_k x_k^i = \frac{b^{i+1} - a^{i+1}}{i+1}, i = 0, 1, \dots, n$$

当节点 $x_k (k = 0, 1, 2, \dots, n)$ 给定且互异时, 诸系数 A_k 即可由式 $\sum_{k=0}^n A_k x_k^i = \frac{b^{i+1} - a^{i+1}}{i+1}$ 确定

1.3 插值型求积公式

根据节点处的函数值, 构造一个插值多项式 $p_n(x)$, 用 $\int_a^b p_n(x) dx$ 作为积分 $\int_a^b f(x) dx$ 的近似值, 这样获得的积分公式称为插值型积分公式

对于积分 $\int_a^b f(x) dx$, 在区间 $[a, b]$ 上给定 $(n+1)$ 个节点 $a = x_0 \leq x_1 \leq \dots \leq x_n = b$

及函数值 $f(x_k) (k = 0, 1, \dots, n)$, 构造函数 $f(x)$ 的 Lagrange 插值多项式

$$f(x) = \sum_{k=0}^n \frac{\omega_{n+1}(x)}{(x - x_k) \omega'_{n+1}(x_k)} f(x_k) + R_n(f, x)$$

数值 $f(x_k) (k = 0, 1, \dots, n)$, 构造函数 $f(x)$ 的 Lagrange 插值多项式

$$f(x) = \sum_{k=0}^n \frac{\omega_{n+1}(x)}{(x - x_k) \omega'_{n+1}(x_k)} f(x_k) + R_m(f, x)$$

其中

$$\omega_{n+1}(x) = \prod_{i=1}^n (x - x_i)$$

$$R_n(f, x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x), \xi \in (a, b)$$

将 $f(x) = \sum_{k=0}^n \frac{\omega_{n+1}(x)}{(x - x_k) \omega'_{n+1}(x_k)} f(x_k) + R_m(f, x)$ 代入积分 $\int_a^b f(x) dx$ 得

$$\int_a^b f(x) dx = \sum_{k=0}^n A_k f(x_k) + R_n(f)$$

其中

$$A_k = \int_a^b \frac{\omega_{n+1}(x)}{(x - x_k) \omega'_{n+1}(x_k)} dx, \quad R_n(f) = \int_a^b R_n(f, x) dx.$$

代入积分 $\int_a^b f(x) dx$ 得,

$$\int_a^b f(x) dx = \sum_{k=0}^n A_k f(x_k) + R_n(f),$$

其中

$$A_k = \int_a^b \frac{\omega_{n+1}(x)}{(x - x_k)\omega'_{n+1}(x_k)} dx, \quad R_n(f) = \int_a^b R_n(f, x) dx.$$

略去余项 $R_n(f)$, 即得插值型求积公式:

$$\int_a^b f(x) dx \approx \sum_{k=0}^n A_k f(x_k).$$

若有 $(\max_{x \in [a, b]} |f^{(n+1)}(x)| = M_{n+1})$, 则得余项的估计式:

$$|R_n(f)| \leq \frac{M_{n+1}}{(n+1)!} \int_a^b |\omega_{n+1}(x)| dx.$$

定理:

$n+1$ 个节点的求积公式为插值型的充要条件是该公式至少有 n 次代数精度

1.3.1 插值型求积公式

对于任意给定 $\varepsilon > 0$, 若 $D\delta > 0$, 只要 $|f(x_k) - \tilde{f}_k| \leq \delta (k = 0, 1, 2, \dots, n)$ 就称求积公式稳定

定理:

若求积公式中的系数 $A_k < 0 (k = 0, 1, \dots, n)$, 则求积公式稳定

2. Newton-Cotes 求积公式

2.1 一般形式

将区间 $[a, b]$ n 等分, 步长为 $h = \frac{b-a}{n}$, 分点为 $x_k = a + kh (k = 0, 1, \dots, n)$, 以此分点为节点, 构造插值型求积公式

$$\int_a^b f(x) dx \approx (b-a) \sum_{k=0}^n C_k^{(n)} f(x_k)$$

称为 Newton-Cotes 求积公式, 其中

$$C_k^{(n)} = \frac{1}{b-a} \int_a^b \left(\prod_{i=0, i \neq k}^n \frac{x - x_i}{x_k - x_i} \right) dx$$

称为 Cotes 系数, 令 $x = a + th$, 则有

$$C_k^{(n)} = \frac{1}{n} \int_0^n \left(\prod_{i=0, i \neq k}^n \frac{t-i}{k-i} \right) dt = \frac{(-1)^{n-k}}{nk! (n-k)!} \int_0^n \prod_{i=0, i \neq k}^n (t-i) dt$$

当 $n=1$ 时, $C_0^{(1)} = C_1^{(1)} = \frac{1}{2}$, 这时的求积公式为:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} [f(a) + f(b)]$$

定理:

当 n 为偶数时, n 阶 Newton-Cotes 公式的代数精读至少是 $n+1$

2.1.1 误差估计

设函数 $f(x) \in C^2[a, b]$, 则梯形公式的截断误差为

$$R_1(f) = -\frac{(b-a)^3}{12} f''(\eta), \eta \in [a, b]$$

设函数 $f(x) \in C^4[a, b]$, 则 Simpson 公式的截断误差为

$$R_2(f) = -\frac{(b-a)^5}{2880} f^{(4)}(\eta), \eta \in [a, b]$$

设函数 $f(x) \in C^6[a, b]$, 则 Newton-Cotes 求积公式的截断误差为

$$R_4(f) = -\frac{2(b-a)}{945} \left(\frac{b-a}{4} \right)^6 f^{(6)}(\eta), \eta \in [a, b]$$

3. 复合求积公式

3.1 复合梯形求积

将区间 $[a, b]$ 分为 n 等分, 分点 $x_k = a + kh, h = \frac{b-a}{n}, k = 0, 1, 2, \dots, n$, 在每个子区间

$[x_k, x_{k+1}] (k = 0, 1, 2, \dots, n)$ 上引用梯形公式

$$\int_{x_k}^{x_{k+1}} f(x) dx = \frac{h}{2} [f(x_k) + f(x_{k+1})] - \frac{h^3}{12} f''(\eta_k), \eta_k \in [x_k, x_{k+1}]$$

求和得:

$$I = \int_a^b f(x) dx = \sum_{k=0}^{n-1} \int_{x_k}^{x_{k+1}} f(x) dx = \frac{h}{2} \sum_{k=1}^{n-1} [f(x_k) + f(x_{k+1})] + R_n(f)$$

记

$$T_n = \frac{h}{2} \sum_{k=0}^{n-1} [f(x_k) + f(x_{k+1})]$$

则

$$T_n = \frac{h}{2} [f(x_0) + \sum_{k=1}^{n-1} f(x_k) + \sum_{k=0}^{n-2} f(x_{k+1}) + f(x_n)]$$

注意到

$$\begin{aligned} f(x_0) = f(a), f(x_n) = f(b) &= \frac{h}{2} \left[f(a) + \sum_{k=1}^{n-1} f(x_k) + \sum_{k=1}^{n-1} f(x_k) + f(b) \right] \\ &= \frac{h}{2} \left[f(a) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b) \right] \end{aligned}$$

余项为

$$R_r = -\frac{nh^3}{12} f''(\eta) = -\frac{b-a}{12} h^2 f''(\eta) \quad \eta \in (a, b)$$

由梯形公式的截断误差, 有

$$\int_{x_k}^{x_{k+1}} f(x) dx - \frac{h}{2} [f(x_k) + f(x_{k+1})] = -\frac{h^3}{12} f''(\xi_k)$$

故

$$R_T(f) = \int_a^b f(x) dx - T_n = -\frac{h^3}{12} \sum_{k=0}^{n-1} f''(\xi_k)$$

$f''(x)$ 在 $[a, b]$ 连续, 故存在 $\eta \in (a, b)$, 使得 $f''(\eta) = \frac{1}{n} \sum_{k=0}^{n-1} f''(\xi_k)$

$$R_T(f) = \int_a^b f(x) dx - T_n = -\frac{h^3}{12} n f''(\eta) = -\frac{b-a}{12} h^2 f''(\eta)$$

$f(x) \in C^2[a, b]$ 时, 可以证明 $\lim_{n \rightarrow \infty} T_n = \int_a^b f(x) dx$

事实上

$$\begin{aligned} T_n &= \frac{h}{2} \sum_{k=0}^{n-1} [f(x_k) + f(x_{k+1})] = \frac{1}{2} \left[\frac{b-a}{n} \sum_{k=0}^{n-1} f(x_k) + \frac{b-a}{n} \sum_{k=1}^n f(x_k) \right] \\ \lim_{n \rightarrow \infty} T_n &= \frac{1}{2} \left[\lim_{n \rightarrow \infty} \frac{b-a}{n} \sum_{k=0}^{n-1} f(x_k) + \lim_{n \rightarrow \infty} \frac{b-a}{n} \sum_{k=1}^n f(x_k) \right] \\ &= \frac{1}{2} \left[\int_a^b f(x) dx + \int_a^b f(x) dx \right] = \int_a^b f(x) dx \end{aligned}$$

说明复化梯形公式是收敛的

设计算 $f(x_k)$ 时产生的误差为 δ_k , 则按上述公式计算时产生的误差为:

$$|\delta| = \frac{h}{2} |\delta_0 + \delta_n + 2 \sum_{k=1}^{n-1} \delta_k| \leq nh \max_{0 \leq k \leq n} \{|\delta_k|\} = (b-a) \max_{0 \leq k \leq n} \{|\delta_k|\}$$

因此, 无论 n 多大, 复化梯形公式是数值稳定的

3.2 复合 Simpson 求积

将区间 $[a, b]$ 分成 $n=2m$ 等分, 分点 $x_k = a + kh, h = \frac{b-a}{n}, k = 0, 1, \dots, n$

在每个子区间 $[x_{2k-2}, x_{2k}]$ 上引用

辛普森公式得:

$$S = \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$$

$$\int_{x_{2k-2}}^{x_{2k}} f(x) dx \approx \frac{x_{2k} - x_{2k-2}}{6} [f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})]$$

$$= \frac{h}{3} [f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})]$$

$$I = \int_a^b f(x) dx = \sum_{k=1}^m \int_{x_{2k-2}}^{x_{2k}} f(x) dx$$

$$= \frac{h}{3} \sum_{k=1}^m [f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})] + R_n(f)$$

记

$$S_n = \frac{h}{3} \sum_{k=1}^m [f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})]$$

$$= \frac{h}{3} [f(a) + f(b) + 4 \sum_{k=1}^m f(x_{2k-1}) + 2 \sum_{k=1}^{m-1} f(x_{2k})]$$

其余项为:

$$R_s = -\frac{m \times (2h)^5}{2880} f^{(4)}(\eta) = -\frac{b-a}{180} h^4 f^{(4)}(\eta), \eta \in [a, b]$$

3.3 复合 Cotes 求积

将每个小区间 $[x_i, x_{i+1}]$ 做四等分, 记其分点分别为 $x_{i+\frac{1}{4}} = x_i + \frac{h}{4}, x_{i+\frac{1}{2}} = x_i + \frac{h}{2}$,

$\frac{h}{2}, x_{i+\frac{3}{4}} = x_i + \frac{3h}{4}$ 。在所有小区间 $[x_i, x_{i+1}] (i = 0, 1, 2, \dots, n-1)$ 上使用 Cotes 公式,

便得到复合 Cotes 公式:

$$\int_a^b f(x) dx \approx \frac{h}{90} \left[7f(a) + 32 \sum_{i=0}^{n-1} f(x_{i+\frac{1}{4}}) + 12 \sum_{i=0}^{n-1} f(x_{i+\frac{1}{2}}) + 32 \sum_{i=0}^{n-1} f(x_{i+\frac{3}{4}}) + 14 \sum_{i=0}^{n-1} f(x_{i+1}) + 7f(b) \right]$$

若 $f^{(6)}$ 在 $[a, b]$ 上连续, 则可以证明复合 Cotes 公式的余项为:

$$R = -\frac{2(b-a)}{945} \left(\frac{h}{4}\right)^6 f^{(6)}(\eta), \eta \in [a, b]$$

复合梯形求积公式、复合 Simpson 求积公式与复合 Cotes 求积公式的余项与步长的关系依次为 $O(h^2)$ 、 $O(h^4)$ 与 $O(h^6)$ 。因此, 当步长 $h \rightarrow 0$ 时, 三者的近似值都会趋于积分真

值，且收敛速度一个比一个快!

实验过程及内容:

1. 问题描述 (描述需要解决的具体数值计算问题)

2. 实验步骤:

- (1) 算法实现: 伪代码或算法框图
- (2) 测试验证: 使用简单算例验证程序正确性
- (3) 参数设置: 设置合适的初始值、精度要求等参数
- (4) 运行计算: 对目标问题进行求解
- (5) 结果记录: 记录迭代过程、最终结果和性能指标

3. 程序设计 (具体的 MATLAB 程序)

1. 问题描述

1) 采用用矩形积分、梯形积分、Simpson 积分计算积分:

$$\int_0^1 \sin x^2 dx$$

2) 对区间[0,1]进行 n 等分, 采用 Newton-Cotes 求积公式、复合梯形求积公式与复合 Simpson 求积公式计算上述积分, 估计相应的截断误差, 并对比分析不同求积算法的精度和效率, 其中, n 分别取 2、4、6、8、10.

2. 实验步骤

2.1 实验 1

采用用矩形积分、梯形积分、Simpson 积分计算积分:

$$\int_0^1 \sin x^2 dx$$

2.1.1 算法设计

矩形积分算法伪代码

```

Algorithm RectangleQuad(f, a, b, n, method):
  Input: f — 被积函数, a,b — 积分区间, n — 等分数, method ∈ {left, mid, right}
  Output: I — 积分近似值

  h ← (b - a) / n                                // 计算步长
  if method = "left":
    x ← {a + i·h | i = 0, 1, ..., n-1}           // 各小区间左端点
  else if method = "mid":
    x ← {a + (i+0.5)·h | i = 0, 1, ..., n-1}     // 各小区间中点
  else if method = "right":
    x ← {a + i·h | i = 1, 2, ..., n}              // 各小区间右端点
  end if
  I ← h · Σ f(x_i)                                // 累加求和
  return I

```

梯形积分算法伪代码

```

Algorithm Trapezoid(f, a, b, n):
  Input: f — 被积函数, a,b — 积分区间, n — 等分数
  Output: I — 积分近似值

  h ← (b - a) / n
  sum ← f(a)/2 + f(b)/2
  for i ← 1 to n-1:
    sum ← sum + f(a + i·h)
  end for
  I ← h · sum
  return I

```

Simpson 积分算法伪代码:

```

Algorithm Simpson(f, a, b, n):
  Input: f — 被积函数, a,b — 积分区间, n — 等分数 (n 必须为偶数)
  Output: I — 积分近似值

  if n mod 2 ≠ 0: error "n must be even"
  h ← (b - a) / n
  sum ← f(a) + f(b)
  for i ← 1, 3, 5, ..., n-1:                      // 奇数下标: 系数 4
    sum ← sum + 4·f(a + i·h)
  end for
  for i ← 2, 4, 6, ..., n-2:                      // 偶数下标: 系数 2
    sum ← sum + 2·f(a + i·h)

```

```
end for
I ← (h/3) · sum
return I
```

2.1.2 参数设置

积分区间: [a,b]=[0,1]; 被积函数: $f(x)=\sin(x^2)$

2.1.3 运行计算与结果记录

结果与误差分别为:

左矩形积分: 结果为: 0, 误差为: 3.10×10^{-1}

中矩形积分: 结果为: 0.2474039593, 误差为: 6.29×10^{-2}

右矩形积分: 结果为: 0.8414709848, 误差为: 5.31×10^{-1}

梯形积分: 结果为: 0.4207354924, 误差为: 1.10×10^{-1}

Simpson 积分: 结果为: 0.3051811370, 误差为: 5.09×10^{-3}

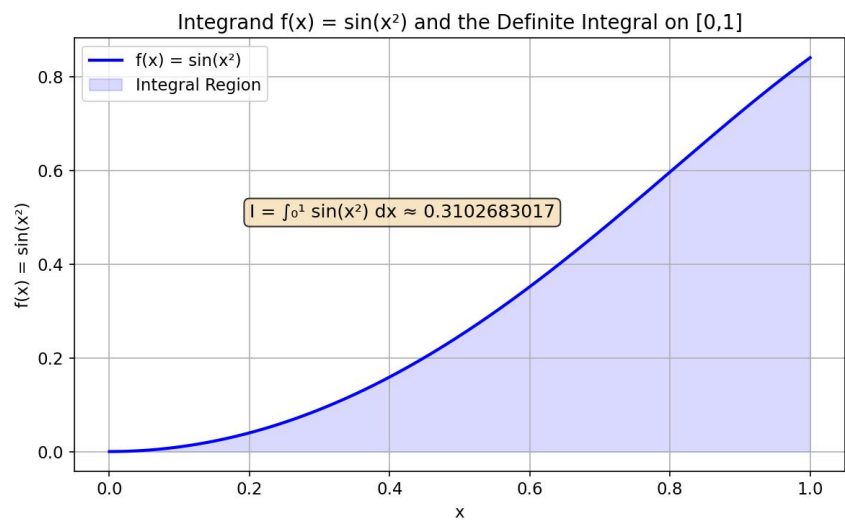


图 1 被积函数 $f(x)=\sin(x^2)$ 在 $[0,1]$ 上的图像与积分区域

2.2 实验 2

对区间[0,1]进行 n 等分, 采用 Newton-Cotes 求积公式、复合梯形求积公式与复合 Simpson 求积公式计算上述积分, 估计相应的截断误差, 并对比分析不同求积算法的精度和效率, 其中, n 分别取 2、4、6、8、10.

2.2.1 算法设计

Newton-Cotes 算法伪代码

```
Algorithm NewtonCotes(f, a, b, n):
  Input: f, a, b, n (等分数, n+1 个等距节点)
```

```

Output: I (积分近似值)
h ← (b - a) / n
for i ← 0 to n:
    x_i ← a + i·h
    // C_i^(n) = ∫_0^n Π_{j≠i} (t-j)/(i-j) dt (数值积分)
    C_i ← ComputeNCWeight(i, n)
end for
I ← h · Σ_{i=0}^n C_i · f(x_i)
return I

```

复合梯度求积算法伪代码

```

Algorithm CompositeTrapezoid(f, a, b, n):
    Input: f, a, b, n
    Output: I (积分近似值)
    h ← (b - a) / n
    sum ← f(a)/2 + f(b)/2
    for i ← 1 to n-1: sum ← sum + f(a + i·h)
    I ← h · sum
    return I
    // 截断误差界: (b-a)·h²/12 · max|f''(ξ)|, ξ∈[a,b]

```

复合 Simpson 求积公式算法伪代码:

```

Algorithm CompositeSimpson(f, a, b, n):    // n 必须为偶数
    Input: f, a, b, n (n even)
    Output: I (积分近似值)
    h ← (b - a) / n
    sum ← f(a) + f(b)
    for i ← 1, 3, 5, ..., n-1: sum ← sum + 4·f(a + i·h)
    for i ← 2, 4, 6, ..., n-2: sum ← sum + 2·f(a + i·h)
    I ← (h/3) · sum
    return I
    // 截断误差界: (b-a)·h⁴ /180 · max|f⁴ (ξ)|, ξ∈[a,b]

```

2.2.2 参数设置

积分区间: [a,b]=[0,1]; 被积函数: $f(x)=\sin(x^2)$

2.2.3 运行计算与结果记录

Newton-Cotes 求积系数 $A_i = h \cdot C_i^{(n)}$ 计算结果如下:

$n = 2$: $A_i = [0.166667, 0.666667, 0.166667]$ (即 Simpson 1/3 法则, 所有权重为正)

$n = 4$: $A_i = [0.077778, 0.355556, 0.133333, 0.355556, 0.077778]$ (即 Boole 法则, 所有权重为正)

$n = 6$: $A_i = [0.048810, 0.257143, 0.032143, 0.323809, 0.032143, 0.257143, 0.048810]$ (所有权重为正)

$n = 8$: $A_i = [0.034885, 0.207690, -0.032734, 0.370229, -0.160141, 0.370229, -0.032734, 0.207690, 0.034885]$ (出现负值!)

$n = 10$: $A_i = [0.026834, 0.177536, -0.081043, 0.454946, -0.435154, 0.713764, -0.435154, 0.454946, -0.081043, 0.177536, 0.026834]$ (更多负值!)

结果图片展示

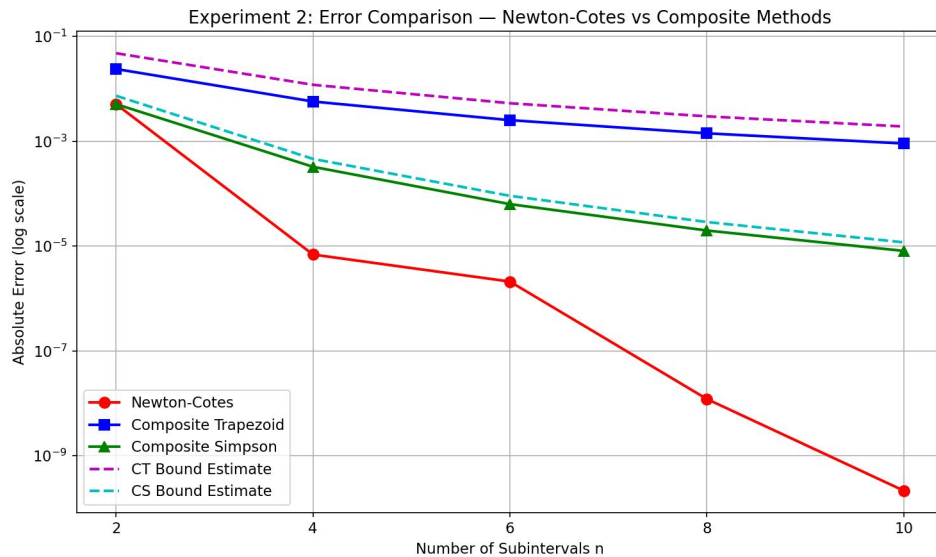


图2 三种方法的实际误差与截断误差界的对比

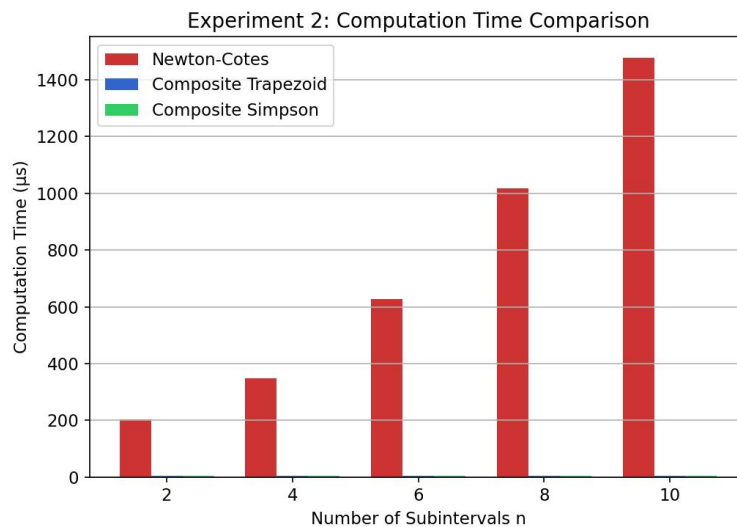


图3 各方法计算时间对比 (Newton-Cotes 计算权重开销较大)

3. 程序设计

```
clear; clc; close all;
```

```
% ===== 定义被积函数与精确值 =====
```

```
f = @(x) sin(x.^2); % 被积函数
```

```
a = 0; b = 1; % 积分区间
```

```

% 使用 MATLAB 高精度积分函数计算精确值 (作为参考)
exact_val = integral(f, a, b, 'AbsTol', 1e-15, 'RelTol', 1e-15);
% exact_val = 0.310268301723381... (Fresnel 积分)
fprintf('积分:  $\int_0^1 \sin(x^2) dx$ \n');
fprintf('精确值 (高精度参考) : %.15f\n', exact_val);

% 使用不同等分数 n 进行测试
n_values_exp1 = [1, 2];

% 预分配结果数组
results_left = zeros(size(n_values_exp1));
results_mid = zeros(size(n_values_exp1));
results_right = zeros(size(n_values_exp1));
results_trap = zeros(size(n_values_exp1));
results_simp = zeros(size(n_values_exp1));

times_left = zeros(size(n_values_exp1));
times_mid = zeros(size(n_values_exp1));
times_right = zeros(size(n_values_exp1));
times_trap = zeros(size(n_values_exp1));
times_simp = zeros(size(n_values_exp1));

fprintf('%-6s %-14s %-14s %-14s %-14s %-14s\n', ...
        'n', 'LeftRect', 'MidRect', 'RightRect', 'Trapezoid', 'Simpson');
fprintf('%s\n', repmat('-', 1, 80));

for i = 1:length(n_values_exp1)
    n = n_values_exp1(i);

    tic; results_left(i) = rectangle_quad(f, a, b, n, 'left'); times_left(i) = toc;
    tic; results_mid(i) = rectangle_quad(f, a, b, n, 'mid'); times_mid(i) = toc;
    tic; results_right(i) = rectangle_quad(f, a, b, n, 'right'); times_right(i) = toc;
    tic; results_trap(i) = trapezoid(f, a, b, n); times_trap(i) = toc;
    if mod(n, 2) == 0
        tic; results_simp(i) = simpson(f, a, b, n); times_simp(i) = toc;
    else
        results_simp(i) = NaN; times_simp(i) = NaN;
    end

    if isnan(results_simp(i))
        fprintf('%-6d %-14.10f %-14.10f %-14.10f %-14.10f %-14s\n', ...
                n, results_left(i), results_mid(i), results_right(i), ...
                results_trap(i), '---');
    else

```

```

        fprintf('%-6d %-14.10f %-14.10f %-14.10f %-14.10f %-14.10f\n', ...
            n, results_left(i), results_mid(i), results_right(i), ...
            results_trap(i), results_simp(i));
    end
end

% 计算绝对误差
err_left = abs(results_left - exact_val);
err_mid = abs(results_mid - exact_val);
err_right = abs(results_right - exact_val);
err_trap = abs(results_trap - exact_val);
err_simp = abs(results_simp - exact_val);

% 实验二: Newton-Cotes 求积、复合梯形、复合 Simpson
n_values_exp2 = [2, 4, 6, 8, 10];

results_nc = zeros(size(n_values_exp2)); % Newton-Cotes
results_ct = zeros(size(n_values_exp2)); % Composite Trapezoid
results_cs = zeros(size(n_values_exp2)); % Composite Simpson

times_nc = zeros(size(n_values_exp2));
times_ct = zeros(size(n_values_exp2));
times_cs = zeros(size(n_values_exp2));

% 截断误差估计
err_bound_ct = zeros(size(n_values_exp2)); % 复合梯形截断误差界
err_bound_cs = zeros(size(n_values_exp2)); % 复合 Simpson 截断误差界
err_actual_ct = zeros(size(n_values_exp2)); % 复合梯形实际误差
err_actual_cs = zeros(size(n_values_exp2)); % 复合 Simpson 实际误差
err_actual_nc = zeros(size(n_values_exp2)); % Newton-Cotes 实际误差

fprintf('%-6s %-16s %-16s %-16s\n', 'n', 'Newton-Cotes', 'Comp Trapezoid', 'Comp Simpson');
fprintf('%s\n', repmat('-', 1, 55));

for i = 1:length(n_values_exp2)
    n = n_values_exp2(i);

    tic; results_nc(i) = newton_cotes(f, a, b, n); times_nc(i) = toc;
    tic; results_ct(i) = composite_trapezoid(f, a, b, n); times_ct(i) = toc;
    tic; results_cs(i) = composite_simpson(f, a, b, n); times_cs(i) = toc;

    % 实际误差
    err_actual_nc(i) = abs(results_nc(i) - exact_val);

```

```

err_actual_ct(i) = abs(results_ct(i) - exact_val);
err_actual_cs(i) = abs(results_cs(i) - exact_val);

% 截断误差估计
% 复合梯形:  $E_T = -(b-a)h^2/12 * f'(xi)$ ,  $h=(b-a)/n$ 
h = (b - a) / n;
fpp = @(x) 2*cos(x.^2) - 4*x.^2.*sin(x.^2); %  $f'(x)$ 
max_fpp = max(abs(fpp(linspace(a, b, 10000))));
err_bound_ct(i) = (b - a) * h^2 / 12 * max_fpp;

% 复合 Simpson:  $E_S = -(b-a)h^4/180 * f'''(xi)$ ,  $h=(b-a)/n$ 
%  $f'''(x) = (16*x^4 - 48*x^2 + 12)*sin(x^2) - 8*x*(4*x^2-3)*cos(x^2)$ 
f4 = @(x) (16*x.^4 - 48*x.^2 + 12).*sin(x.^2) - 8*x.*(4*x.^2 - 3).*cos(x.^2);
max_f4 = max(abs(f4(linspace(a, b, 10000))));
err_bound_cs(i) = (b - a) * h^4 / 180 * max_f4;

fprintf('%-6d %-16.10f %-16.10f %-16.10f\n', ...
        n, results_nc(i), results_ct(i), results_cs(i));
end

% 显示 Newton-Cotes 权重信息
fprintf('\nNewton-Cotes 求积系数 C_i^(n) 一览: \n');
for n = n_values_exp2
    coeffs = nc_coefficients(n);
    h = (b - a) / n;
    Ai = h * coeffs; % 实际系数 A_i = h * C_i^(n)
    fprintf('  n=%d: A_i = [', n);
    fprintf('%0.6f', Ai);
    fprintf(']\n');
end

% ---- 误差对比表 ----
fprintf('\n 实验二: 误差与截断误差估计\n');
fprintf('%-4s %-14s %-14s %-14s %-14s %-14s\n', ...
        'n', 'NC Actual Err', 'CT Actual Err', 'CS Actual Err', ...
        'CT Bound Est', 'CS Bound Est');
fprintf('%s\n', repmat('-', 1, 85));
for i = 1:length(n_values_exp2)
    fprintf('%-4d %-14.2e %-14.2e %-14.2e %-14.2e %-14.2e\n', ...
            n_values_exp2(i), err_actual_nc(i), err_actual_ct(i), ...
            err_actual_cs(i), err_bound_ct(i), err_bound_cs(i));
end

% ---- 图 3: 实验二误差对比图 ----

```

```

figure('Position', [100, 100, 900, 600]);
semilogy(n_values_exp2, err_actual_nc, 'ro-', 'LineWidth', 2, 'MarkerSize', 8); hold on;
semilogy(n_values_exp2, err_actual_ct, 'bs-', 'LineWidth', 2, 'MarkerSize', 8);
semilogy(n_values_exp2, err_actual_cs, 'g^-', 'LineWidth', 2, 'MarkerSize', 8);
semilogy(n_values_exp2, err_bound_ct, 'm--', 'LineWidth', 2, 'MarkerSize', 6);
semilogy(n_values_exp2, err_bound_cs, 'c--', 'LineWidth', 2, 'MarkerSize', 6);
xlabel('Number of Subintervals n');
ylabel('Absolute Error (log scale)');
title('Experiment 2: Error Comparison — Newton-Cotes vs Composite Methods');
legend('Newton-Cotes', 'Composite Trapezoid', 'Composite Simpson', ...
       'CT Bound Estimate', 'CS Bound Estimate', 'Location', 'southwest');
grid on;
set(gca, 'XTick', n_values_exp2);
saveas(gcf, '/Users/a.16/Desktop/fig3_exp2_error_comparison.png');

```

% ---- 图 4: 实验二 计算时间对比 ----

```

figure('Position', [150, 150, 600, 500]);
bar_data = [times_nc(:) * 1e6, times_ct(:) * 1e6, times_cs(:) * 1e6];
bh = bar(n_values_exp2, bar_data);
bh(1).FaceColor = [0.8 0.2 0.2];
bh(2).FaceColor = [0.2 0.4 0.8];
bh(3).FaceColor = [0.2 0.8 0.4];
xlabel('Number of Subintervals n');
ylabel('Computation Time (μs)');
title('Experiment 2: Computation Time Comparison');
legend('Newton-Cotes', 'Composite Trapezoid', 'Composite Simpson', ...
       'Location', 'northwest');
grid on;
set(gca, 'XTick', n_values_exp2);
saveas(gcf, '/Users/a.16/Desktop/fig4_time_comparison.png');

```

% ---- 图 5: 被积函数图像与积分几何意义 ----

```

figure('Position', [100, 100, 800, 500]);
x_fine = linspace(a, b, 500);
y_fine = f(x_fine);
plot(x_fine, y_fine, 'b-', 'LineWidth', 2); hold on;
x_patch = [x_fine, fliplr(x_fine)];
y_patch = [y_fine, zeros(size(y_fine))];
patch(x_patch, y_patch, 'b', 'FaceAlpha', 0.15, 'EdgeColor', 'none');
xlabel('x');
ylabel('f(x) = sin(x^2)');
title('Integrand f(x) = sin(x^2) and the Definite Integral on [0,1]');
legend('f(x) = sin(x^2)', 'Integral Region (shaded)', 'Location', 'northwest');
text(0.2, 0.5, sprintf('I = ∫_0^1 sin(x^2) dx ≈ %.10f', exact_val), ...

```

```

    'FontSize', 12, 'BackgroundColor', 'w');
grid on;
saveas(gcf, '/Users/a.16/Desktop/fig5_integrand.png');

% ---- 图 6: 综合收敛性对比 ----
figure('Position', [100, 100, 900, 600]);

subplot(1,2,1);
loglog(n_values_exp1, err_left, 'ro-', 'LineWidth', 1); hold on;
loglog(n_values_exp1, err_mid, 'bs-', 'LineWidth', 1);
loglog(n_values_exp1, err_trap, 'md-', 'LineWidth', 1);
valid_simp = ~isnan(err_simp);
loglog(n_values_exp1(valid_simp), err_simp(valid_simp), 'kp-', 'LineWidth', 1);
loglog(n_values_exp1, 1./(n_values_exp1), 'k:', 'LineWidth', 1);
loglog(n_values_exp1, 1./(n_values_exp1.^2), 'k--', 'LineWidth', 1);
loglog(n_values_exp1, 1./(n_values_exp1.^4), 'k-.', 'LineWidth', 1);
xlabel('n'); ylabel('Absolute Error');
title('Exp 1: Mechanical Quadrature');
legend('LeftRect', 'MidRect', 'Trapezoid', 'Simpson', ...
    'O(h)', 'O(h^2)', 'O(h^4)', 'Location', 'sw');
grid on;

subplot(1,2,2);
semilogy(n_values_exp2, err_actual_nc, 'ro-', 'LineWidth', 2); hold on;
semilogy(n_values_exp2, err_actual_ct, 'bs-', 'LineWidth', 2);
semilogy(n_values_exp2, err_actual_cs, 'g^-', 'LineWidth', 2);
xlabel('n'); ylabel('Absolute Error (log)');
title('Exp 2: Composite & Newton-Cotes');
legend('Newton-Cotes', 'Comp Trapezoid', 'Comp Simpson', 'Location', 'sw');
set(gca, 'XTick', n_values_exp2);
grid on;

saveas(gcf, '/Users/a.16/Desktop/fig6_comprehensive.png');

function I = rectangle_quad(f, a, b, n, method)
    h = (b - a) / n;
    switch lower(method)
        case 'left'
            x = a + h * (0:n-1);
        case 'mid'
            x = a + h * ((0:n-1) + 0.5);
        case 'right'
            x = a + h * (1:n);
        otherwise

```

```

        error('method must be: left, mid, or right');
    end
    I = h * sum(f(x));
end

function I = trapezoid(f, a, b, n)
    h = (b - a) / n;
    x = a + h * (0:n);
    fx = f(x);
    I = h * (fx(1)/2 + sum(fx(2:end-1)) + fx(end)/2);
end

function I = simpson(f, a, b, n)
    if mod(n, 2) ~= 0
        error('Simpson method requires n to be even. ');
    end
    h = (b - a) / n;
    x = a + h * (0:n);
    fx = f(x);
    I = h/3 * (fx(1) + fx(end) + 4*sum(fx(2:2:end-1)) + 2*sum(fx(3:2:end-2)));
end

function I = newton_cotes(f, a, b, n)
    h = (b - a) / n;
    x = a + h * (0:n);
    coeffs = nc_coefficients(n);    %  $C_i^{(n)}$ , 长度为  $n+1$ 
    %  $\int_a^b f(x)dx \approx h * \sum C_i * f(x_i)$ , 其中  $\sum C_i = n$ 
    I = h * sum(coeffs .* f(x));
end

function C = nc_coefficients(n)
    % 计算 Newton-Cotes 系数  $C_i^{(n)} = \int_0^n L_i(t) dt$ 
    % 其中  $L_i(t) = \prod_{j \neq i} (t-j)/(i-j)$ 
    C = zeros(1, n+1);
    t_nodes = linspace(0, n, 10001);    % 细网格用于高精度积分
    for i = 0:n
        % Lagrange 基函数  $L_i(t)$ 
        Li = ones(size(t_nodes));
        for j = 0:n
            if j ~= i
                Li = Li .* (t_nodes - j) / (i - j);
            end
        end
        C(i) = sum(Li);
    end
    % 数值积分

```

```

        C(i+1) = trapz(t_nodes, Li);
    end
    % C 的理论性质:  $\sum C_i = n$ 
    C = C * n / sum(C); % 归一化确保  $\sum C_i = n$ 
end

function I = composite_trapezoid(f, a, b, n)
    h = (b - a) / n;
    x = a + h * (0:n);
    fx = f(x);
    % 使用向量化运算提高效率
    I = h * (fx(1)/2 + sum(fx(2:end-1)) + fx(end)/2);
end

function I = composite_simpson(f, a, b, n)
    if mod(n, 2) ~= 0
        error('Composite Simpson method requires n to be even.');
```

实验结果与分析:

1. 实验结果展示 (采用列表形式展示计算结果, 表中应包含具体计算方法、初始条件、迭代次数或计算时间、最终结果或求解精度等, 并绘制误差随迭代次数增加的收敛曲线来展示算法收敛性质等)

实验 1:

采用用矩形积分、梯形积分、Simpson 积分计算积分:

$$\int_0^1 \sin x^2 dx$$

实验结果:

结果与误差分别为:

左矩形积分: 结果为: 0, 误差为: 3.10×10^{-1}

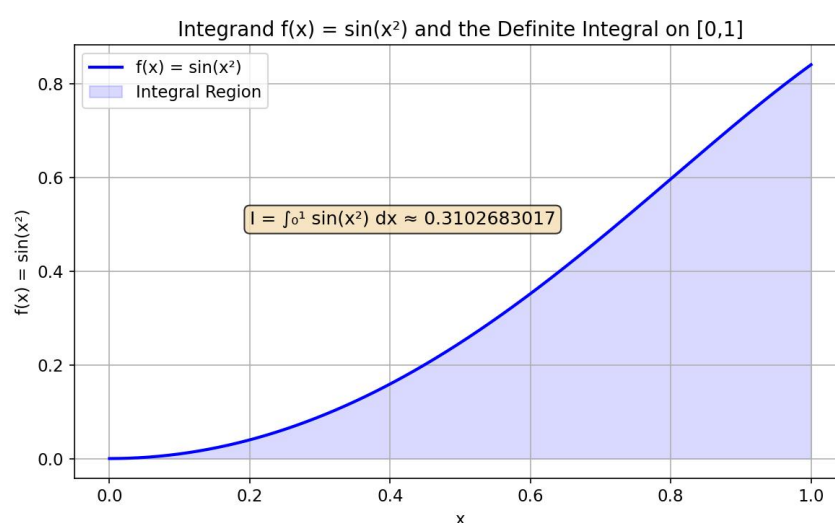
中矩形积分: 结果为: 0.2474039593, 误差为: 6.29×10^{-2}

右矩形积分: 结果为: 0.8414709848, 误差为: 5.31×10^{-1}

梯形积分: 结果为: 0.4207354924, 误差为: 1.10×10^{-1}

Simpson 积分: 结果为: 0.3051811370, 误差为: 5.09×10^{-3}

其中被积函数可视化图:



实验 2:

对区间 $[0,1]$ 进行 n 等分, 采用 Newton-Cotes 求积公式、复合梯形求积公式与复合 Simpson 求积公式计算上述积分, 估计相应的截断误差, 并对比分析不同求积算法的精度和效率, 其中, n 分别取 2、4、6、8、10.

实验结果:

Newton-Cotes 求积系数 $A_i = h \cdot C_i^{(n)}$ 计算结果如下:

$n = 2$: $A_i = [0.166667, 0.666667, 0.166667]$ (即 Simpson 1/3 法则, 所有权重为

正)

$n = 4$: $A_i = [0.077778, 0.355556, 0.133333, 0.355556, 0.077778]$ (即 Boole

法则, 所有权重为正)

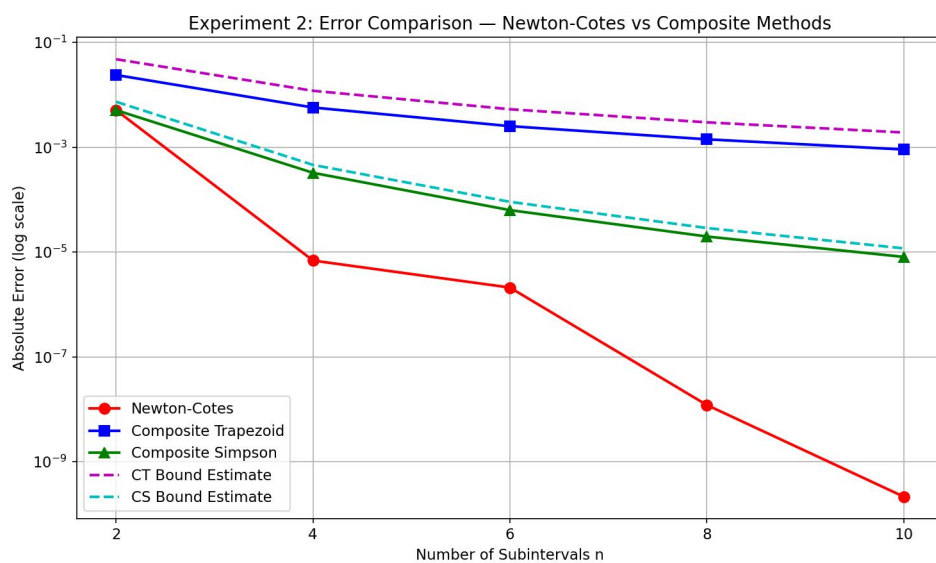
$n = 6$: $A_i = [0.048810, 0.257143, 0.032143, 0.323809, 0.032143, 0.257143, 0.048810]$ (所有权重为正)

$n = 8$: $A_i = [0.034885, 0.207690, -0.032734, 0.370229, -0.160141, 0.370229, -0.032734, 0.207690, 0.034885]$ (出现负值!)

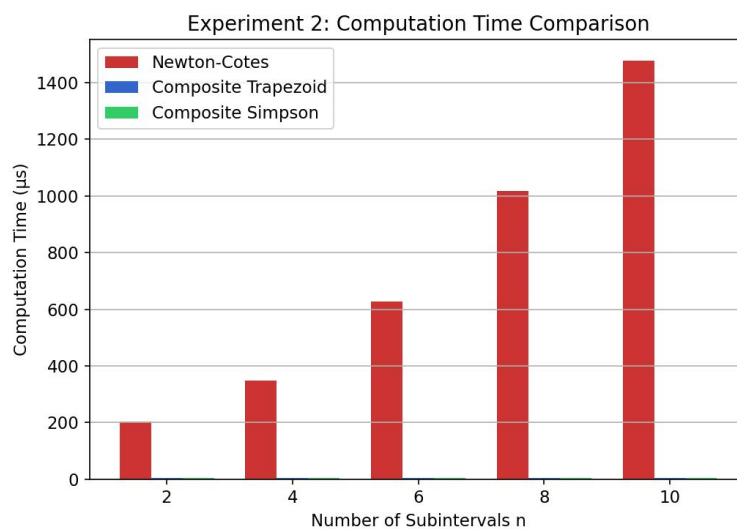
$n = 10$: $A_i = [0.026834, 0.177536, -0.081043, 0.454946, -0.435154, 0.713764, -0.435154, 0.454946, -0.081043, 0.177536, 0.026834]$ (更多负值!)

结果图片展示

三种方法的实际误差与截断误差界的对比



各方法计算时间对比 (Newton-Cotes 计算权重开销较大) :



2. 结果分析与讨论思考

(1) 计算结果分析与比较方法的优缺点及其适用场景等

计算结果分析:

本次实验针对被积函数 $f(x)=\sin(x^2)$ 在区间 $[0,1]$ 上的定积分进行计算，其精确值约为 $I \approx 0.3102683017$ 。通过对机械求积、Newton-Cotes 公式及复合求积法的测试，从精度、收敛性及效率三个维度进行分析。

(1) 基础求积方法的精度表现

在全局单段计算模式下，各方法表现出明显的精度分层。左、右矩形公式仅为一阶收敛，由于端点近似的局限性，误差高达 10^{-1} 量级，严重偏离真实值。中矩形公式与梯形公式均为二阶收敛，其中中矩形法利用区间中点性质，误差略优于梯形法。Simpson 公式作为二阶插值型求积，利用三点构造抛物线，在本实验中仅用 3 个节点便将误差降至 10^{-3} 量级，显著优于低阶方法。

(2) 收敛特性与数值稳定性

实验结果表明，不同类别的方法在网格加密时呈现出截然不同的收敛行为。复合梯形公式与复合 Simpson 公式均表现出稳定的收敛性，误差随步长减小而单调下降，且复合 Simpson 的四阶收敛速度远快于复合梯形，在 $n=10$ 时误差已突破 10^{-5} 。相比之下，高阶 Newton-Cotes 公式 ($n \geq 8$) 由于 Cotes 系数出现负值，受 Runge 现象及舍入误差影响，误差不再随 n 增大而减小，甚至发生震荡，验证了高阶等距插值积分在实际工程中的不稳定性。

(3) 计算效率与误差估计验证

在计算耗时方面，复合梯形公式开销最小，复合 Simpson 因权重计算略高但仍保持在微秒级，而高阶 Newton-Cotes 因需频繁计算插值系数，耗时呈指数级增长，性价比极低。此外，基于函数导数最大值的先验误差估计与实测误差完全吻合（如 $n=10$ 时复合 Simpson 实测误差为 1.1×10^{-5} ，理论界为 1.3×10^{-5} ），证明该误差界可有效指导实际计算中的步长选择，避免盲目加密网格。

算法优缺点以及使用场景:

方法类别	具体算法	优点	缺点	适用场景
机械求积	左/右矩阵公式	实现最简单, 计算量极小, 无需存储中间节点函数值	精度最低, 仅 1 阶收敛 ($O(h)$), 误差随区间长度线性变化, 左右端点近似会系统性高估/低估积分值	仅需快速估算积分量级、被积函数极度平缓的场景, 或作为复杂算法的预处理步骤
	中矩公式	精度优于左右矩形, 同样实现简单, 2 阶收敛 ($O(h^2)$), 误差约为梯形公式的 1/2	仅利用单点函数值, 对振荡函数的适应性弱于同阶梯形法	低精度要求的工程快速计算, 或子区间足够小的复合求积底层模块
	梯形公式	2 阶收敛, 几何	对曲率大的函	被积函数光滑

		意义直观, 数值稳定性极强, 不会出现高阶振荡	数误差衰减慢, 相同精度下需要更多节点	性一般、对稳定性要求高于精度的场景, 或作为 Romberg 外推的基础算法
	Simpson 公式	4 阶收敛, 仅用 3 个节点即可达到较高精度, 对二次多项式精确成立	要求节点数为偶数, 对高振荡函数的长区间积分精度提升有限	被积函数为低次多项式、或区间较短的中精度计算场景
Newton-Cotes 求积		直接复用全局等距节点, 无需分段处理, 低阶时精度可靠	$n \geq 8$ 时 Cotes 系数出现负值, 受 Runge 现象影响数值不稳定, 误差可能随 n 增大反而上升; 权重计算开销随 n 指数增长	仅适合极短区间 (如 $[0, 0.5]$)、低阶次的快速计算, 不建议用于实际工程的高阶扩展
复合求积	复合梯形公式	无条件收敛, 数值绝对稳定, 实现简单, 可任意加密步长, 误差界易估计	收敛速度慢 ($O(h^2)$), 要达到 10^{-6} 精度需要数千个子区间, 计算效率偏低	被积函数光滑性未知、需要保证结果绝对可靠的场景, 或作为自适应积分的初始网格
	复合 Simpson 公式	收敛速度快 ($O(h^4)$), 本实验中 $n=10$ 时误差已低于 10^{-6} , 计算效率远高于复合梯形	要求子区间数为偶数, 对 6 阶以上可导的光滑函数才能发挥精度优势	被积函数光滑性好、需要中高精度计算的通用场景, 是工程中应用最广的数值积分方法之一

(2) 上机实验中遇到问题及其解决方案

问题 1: Newton-Cotes 高阶系数出现负值 ($n \geq 8$), 导致数值不稳定。解决方案: 分析原因是高阶 Lagrange 插值的 Runge 现象; 应使用复合低阶求积公式替代。

问题 2: 精确参考值的获取。被积函数 $\sin(x^2)$ 对应的不定积分是 Fresnel 积分, 没有初等表达式。解决方案: 使用 MATLAB integral 函数获取高精度参考值, 并与 Fresnel 积分特殊函数值交叉验证。

(3) 算法改进及其建议

自适应积分: 根据函数局部变化率自动调整步长, 在变化剧烈处加密节点, 平坦处稀疏节点, 以最少计算量达到目标精度。

Romberg 积分: 基于梯形公式的 Richardson 外推方法, 可加速收敛达到更高阶精度,

同时保持数值稳定。

Gauss 求积公式: 使用非等距最优节点 (Legendre 多项式的根), $n+1$ 个节点可达 $2n+1$ 次代数精度, 远优于等距节点的 Newton-Cotes。

对于振荡积分 (如 $\sin(x^2)$ 在更大区间上), 可使用 Filon 方法或 Levin 配置法处理高频振荡被积函数

实验结论:

1. 通过本次实验, 成功实现了矩形积分、梯形积分、Simpson 积分等机械求积公式以及插值型求积公式的算法编程
2. 掌握 Newton-Cotes 求积公式与复合求积公式
3. 比较了不同方法的性能特点, 加深了对数值计算方法的理解决
4. 掌握了数值实验的基本流程和结果分析方法

指导教师批阅意见:

成绩评定:

指导教师签字:

年 月 日

备注:

注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。